
Bench

Product Requirements

User flows for agent creation, sharing and monitoring

VERSION 1.0 · 15 SEPTEMBER 2026

An internal agent platform for business teams: a subject-matter expert builds an agent once, their team installs it, and it runs from the next trigger.

Written to be built from. Every requirement carries its evidence — from the Plum brief and prototype, and from a four-day founder teardown of a shipping competitor.

Companion artefacts

[versions/v4/bench.html](#) — the clickable prototype

[work/](#) — the evidence digest, gap analysis and decisions log

CONTENTS

0 • How to read this	5
0.1 What this document is	5
0.2 Where the requirements come from	5
0.3 Requirement language and IDs	5
0.4 Evidence markers	5
0.5 What is deliberately out of scope	6
1 • The product in one page	7
2 • The ten laws	8
3 • Roles and permissions	9
3.1 The three roles	9
3.2 Permissions matrix	9
4 • Object model	11
4.1 Agent (extended)	11
4.2 Run (rewritten)	11
4.3 Trigger (new)	13
4.4 Budget (new)	13
4.5 AgentProposal and SelfModPolicy (new)	13
4.6 Unchanged from the prototype	14
5 • Flow A — Creating an agent	15
5.1 A1 • Create	15
5.2 A2 • Describe	16
5.3 A3 • Configure	16
5.4 A4 • The trigger, in a sentence	16
5.5 A5 • Autonomy and guardrails	17
5.6 A6 • What users can change	17
5.7 A7 • The outcome metric	18
5.8 A8 • Test, and try it for real	18
5.9 A9 • Publish	19
5.10 Edge cases — creation	20
6 • Flow B — Sharing an agent	21

6.1 The finding this flow exists to invert	21
6.2 B1 · Audience	21
6.3 B2 · The store	22
6.4 B3 · Preview before consent	22
6.5 B4 · Scope consent, one source at a time	22
6.6 B5 · Install semantics	23
6.7 B6 · Uninstall and requests	23
6.8 B7 · Cross-team spread	24
6.9 B8 · The deprecation contract	24
6.10 Edge cases — sharing	25
7 · Flow C — Monitoring	26
7.1 C1 · Run status	26
7.2 C2 · When a run stops: the Ask	26
7.3 C3 · When a run fails	27
7.4 C4 · The run log	27
7.5 C5 · The trace	28
7.6 C6 · Schedules	28
7.7 C7 · Budgets	28
7.8 C8 · The cost unit	29
7.9 C9 · Adoption and outcome	29
7.10 C10 · The pulse	30
7.11 Edge cases — monitoring	31
8 · Cross-cutting requirements	32
8.1 Notifications	32
8.2 Approvals	32
8.3 Credentials and scopes	33
8.4 Audit	33
8.5 Business context	33
8.6 Self-modification	34
8.7 Accessibility	34
8.8 Mobile	34
9 · Non-functional requirements	35

10 · Anti-requirements	36
11 · Open decisions	37
12 · Acceptance criteria	38
12.1 Creation	38
12.2 Sharing	38
12.3 Monitoring	39
12.4 The pilot test	39
Appendix A · Evidence table	40
Appendix B · Screen inventory	43
Member routes	43
Team admin routes	43
Super admin routes	44
Developer handoff routes	44
Appendix C · Glossary	45

0 · How to read this

0.1 What this document is

Bench exists as a prototype in three versions. It is good at screens and silent on operations: it draws what an agent looks like and not what happens when one fails at 07:02 on a Monday. This document closes that gap and is written to be built from.

It specifies three flows end to end — **creating an agent**, **sharing an agent**, and **monitoring agents in production** — with the screens, the states, the edge cases, the object model and the acceptance criteria for each.

0.2 Where the requirements come from

Two sources, and they are held to different standards.

The Plum brief and the prototype. The thesis, the personas, the insurance-broker domain, and every decision already settled in `versions/v1-v3` and their `docs/`. These are the product's own positions and are carried forward unless something below overturns them.

A founder teardown of a shipping competitor. In September 2026 an operator took a real account on a live agent platform and drove it as a non-ideal customer would for four compressed days: built agents, armed schedules, shared them to an org, broke them deliberately, and recorded every observation against a screenshot. That record — the running teardown, a five-session journal, a 162-row screenshot manifest, and a 27-platform comparison built alongside it — is the closest thing available to watching a real user meet a real product. It lives at `~/Desktop/code/gumloop`.

Its findings are treated as **evidence about one product observed once**, not as market truth. Where a finding is a single observation it says so. Where it contradicts the prototype, this document says which one wins and why.

0.3 Requirement language and IDs

MUST — required for the first production release. Absence is a bug. **SHOULD** — required unless there is a stated reason not to; a deliberate omission needs a decision, not silence. **MAY** — optional; specified so that if it is built, it is built this way.

IDs run `BR-<flow>-<nnn>`: `BR-C-001` for creation, `BR-S-001` for sharing, `BR-M-001` for monitoring, `BR-X-001` for cross-cutting, `BR-N-001` for non-functional. IDs are permanent — a withdrawn requirement is marked withdrawn, never reused.

0.4 Evidence markers

Each requirement that comes from the teardown carries a marker:

MARKER	MEANING
[obs]	Directly observed in the competitor's product, with a screenshot reference
[obs·n=1]	Observed once. Directionally useful, not a base rate.
[msr]	Measured, with the figure quoted
[voc]	From the competitor's public forum — user complaints and requests, with topic counts
[cmp]	From the 27-platform comparison
[jdg]	Judgement. No evidence either way; the reasoning is in <code>work/03-decisions-log.md</code> .

Anything unmarked follows from the prototype's own decisions.

0.5 What is deliberately out of scope

The agent runtime's internals — model choice, tool-calling, context assembly. This document specifies what the runtime must *report*, not how it works.

Billing and invoicing. There is a meter, a projection and caps; there is no price card, no invoice and no overage flow. §12 carries the pricing fork.

Mobile layout beyond the three surfaces named in §9.6.

Cross-tenant distribution. §12 carries it as an open commercial decision.

1 • The product in one page

Employees get individual AI subscriptions. A few make excellent use of them. Most never adopt. The spend is high, the return is invisible, and the way the good users work never reaches anybody else.

Bench replaces the unit. The person with the most domain expertise — who has already built this for themselves — publishes an agent; their team installs it and gets value from the next trigger. Effort concentrates on building one agent well. Everyone else uses it.

Three roles. A **member** installs, changes the input parameters the author exposed, and files requests. A **team admin** authors, tests, publishes and maintains. A **super admin** sets connections, boundaries and budgets across the org and owns the change-management story.

Four things make it a product rather than a prompt library.

1. **An outcome metric is required to publish.** Not a dashboard afterthought — a field that blocks the publish button. No shipping platform does this.
2. **The author decides which knobs are safe to turn.** Members cannot fork. Pressure goes into requests, which are linked to the version that ships them, and release notes name the person who asked.
3. **Test scenarios are generated from the agent's own config,** guardrails generate their own adversarial cases, and a failing guardrail case blocks publish.
4. **Every team's business context is a first-class object** — how it works, its targets, its incentives, what separates a good performer from a weak one — and it attaches to that team's agents automatically.

What v4 adds, and what this document specifies. All of the above describes an agent at rest. This document is about agents in motion: what a run is, what happens when one stops, who finds out, what it costs, who owns it after it is published, and how a team of fourteen people running eight agents knows whether today went well.

2 · The ten laws

Every requirement in this document is consistent with these. Where a requirement seems to conflict with a law, the law wins and the requirement is wrong.

L1 · An object exists before it is configured. Create makes a live draft in one click. No wizard, no naming modal, nothing to finish before there is something to react to. **[obs]** Observed: "Create Agent does not open a wizard — it instantly creates a working agent and drops you in a full workspace. Zero-to-agent in ~12 seconds, before I had told it anything."

L2 · Every empty state names what goes there and whose move it is. The prototype already does this well. The teardown confirms why it matters: a fresh agent workspace that suggests four next steps in the product's own voice teaches the object model while selling the product. **[obs]**

L3 · Cost is shown before the action, never only after. Any control that commits recurring spend states the projection at the moment of commitment. **[obs]** The one surface in the competitor that does this — a pre-sync panel reading "estimated cost ~5 credits" — is described in the teardown as the pattern every credit-consuming surface should follow.

L4 · One number, one source. Every surface showing cost, count or status reads the same ledger. **[obs·n=1]** In the competitor, the trigger drawer's credit counter undercounted the task ledger by exactly 3 on all three triggers measured (18 vs 21, 100 vs 103, 56 vs 59).

L5 · A run that did no work is never called completed. **[obs]** Two runs that could not do their job — one stopped for a missing input, one hit a hard block — both recorded as *Completed*, one of them at 25 credits.

L6 · Anything that can be created in one click can be stopped in one click, from a labelled control. **[obs]** Creating a trigger was one visible click. Deleting one took six passes: the control is an unlabelled ellipsis inside a drawer, with no `aria-label`, invisible to keyboard and screen-reader navigation.

L7 · Nothing bills that does not appear as a run. No background meters. **[msr]** The competitor's inbox classifier bills once every one to three hours whether the product is opened or not.

L8 · An agent may propose; only a human publishes. **[jdg]**, inverting **[obs]**: that product ships Allow Self-Updates, AI-managed triggers, AI connector discovery, AI skill editing, self-cloning sub-agents and tool discovery on AUTO — four AI-managed subsystems on one panel, on by default.

L9 · Publishing transfers custody, not just permission. **[obs]** The teardown's clearest structural failure — see §7.1.

L10 · Platform vocabulary never reaches a member's screen. No "eval", "grader", "dataset", "credit", "token", "trace", "run id". Members get situations, outcomes and plain sentences. Carried from v2 and reinforced: the competitor's most useful cost surface is the one almost nobody finds.

3 · Roles and permissions

3.1 The three roles

Member — the person the product is for. Installs agents, sets the parameters the author exposed, acts on what agents produce, approves or bins drafts, answers questions agents raise, and files requests. Cannot author, cannot fork, cannot see anyone else's numbers.

Team admin — the subject-matter expert who builds. Authors and publishes agents for one team, owns the requests inbox, sets budgets on their own agents, reads their team's run log, maintains what they published. In the pilot this is Karan Mehta, Sales Lead · West.

Super admin — owns the org. Creates teams, grants connections and scopes, sets org and team budgets, sets the self-modification policy, reads everything, and owns the change management story. In the pilot, Ananya Rao, VP Revenue Operations.

A person holds exactly one role per team and may be a member of several teams. A team admin is a member of their own team for everything they install.

3.2 Permissions matrix

BR-X-001 The system **MUST** enforce the following:

CAPABILITY	MEMBER	TEAM ADMIN	SUPER ADMIN
Install an agent for themselves	Yes	Yes	Yes
Change exposed input parameters	Yes	Yes	Yes
Change instructions, scopes, guardrails	No	Own team	Any team
Publish to a team	No	Own team	Any team
Set an agent to auto-act	No	Request only	Yes
Grant a new data source	No	Request only	Yes
See their own outcome metrics	Yes	Yes	Yes
See another person's usage	No	Own team	Any team
Open a run trace	Own runs	Own team	Everything
Answer a blocked run	Own runs	Own team	Any
Pause or stop a schedule	No	Own team	Any team
Set a budget cap	No	Own agents	Org and teams
Take another team's agent as a template	No	Own team	Any team
Approve an agent's proposed change	No	Own team	Any team
Change the self-modification policy	No	No	Yes
Retire an agent	No	Own team	Any team
Write business context	No	Own team	Any team
Read the audit log	Own runs	Own team	Everything

BR-X-002 A member **MUST NOT** be able to fork an agent by any route. [jdg] The alternative — every user keeping a private variant — is the fragmentation the product exists to prevent. Pressure is routed into requests instead (§7.7).

BR-X-003 Run *content* — the text of a draft, the items in a digest, a thread — **MUST** be visible only to the person the run was for, plus anyone that person explicitly shares it with. Run *metadata* — status, duration, cost band, step labels — is visible to the team admin and super admin. [jdg] An admin who can read their team's inboxes through the analytics screen is a different product with a different sale.

4 • Object model

The prototype defines seven objects. This adds six and rewrites one. Field names are the API contract.

4.1 Agent (extended)

```
Agent {
  id, name, blurb, long_description, category, team_id | ORG
  verified, created_at

  ownership {
    author_id           // permanent credit. never changes.
    owner_team_id       // set on FIRST PUBLISH. custody moves here.
    maintainer_id       // defaults to author; reassignable;
                       // forced on author deactivation
    forked_from? { agent_id, team_id, version, at }
  }

  triggers      [ Trigger ]
  scopes        [ { source, access: read|write, required: bool,
                    plain_language: string, granted_by_installer: bool } ]
  instructions   text           // inherits team business_context
  attachments    [ glossary_id, doc_id... ]

  autonomy      suggest | draft | auto_act
  guardrails    [ { rule, generates_tests: true } ]
  auto_act_limits { ceiling?, allowed_actions[], owner_id, written_reason }
  approval_rules [ ApprovalRule ]           // NEW — see §8.6

  input_schema  [ InputParam ]
  locked_notes  [ string ]

  outcome       Outcome           // REQUIRED to publish
  budget        Budget            // REQUIRED to publish
  self_mod      SelfModPolicy     // inherited from org, per-agent override

  version       semver, changelog, published_at, published_by
  status        draft | in_test | awaiting_review | live | retired
}
```

4.2 Run (rewritten)

The prototype had `Run` with a `trace[]` and a free-text `status`. Both change.

```

Run {
  id, installation_id, agent_version, trigger_id, trigger_reason
  started_at, duration_ms

  status    completed | nothing_to_do | blocked | failed | partial

  cost_band    light | standard | heavy
  read         [ source ]          // resolved at run time, not declared
  wrote        [ artifact_ref ]
  outcome_line string              // ONE line. shown on the row.
  trace        [ RunStep ]
  items        [ RunItem ]
  ask?         Ask                 // set iff status = blocked
  failure?     Failure             // set iff status = failed
  pinned       bool                // exempts the trace from expiry
}

RunStep {
  seq, label, detail, at, duration_ms
  kind    ok | warn | fail | auto | block
          // warn = a decision NOT to act: a missing source, a declared
          // unknown, a refusal, a suppressed write.
}

Ask {
  question, why_not_decided, assignee_id, options[]
  raised_at, escalates_at    // = the agent's next scheduled run
  answered_at?, answer?     // answering RESUMES; it never restarts
}

Failure {
  cause, what_it_tried, suggested_fix
  notified [ user_id ]       // maintainer, immediately
}

RunItem {
  id, severity, title, detail
  action_kind    none | draft | auto_acted
  artifact_ref?
  user_response  acted | dismissed | ignored | edited
  ignored_at?    // computed at the agent's next run; 72h for on-demand
}

```

4.3 Trigger (new)

```
Trigger {
  id, agent_id, label
  kind          on_demand | schedule | event | threshold
  natural_text   string      // exactly what the author typed
  parsed        { cron? | event_source? | threshold_expr? }
  parse_echo    string      // rendered back to the author before save
  projected_per_day
  projected_units_per_month // shown BEFORE save. L3.
  state         on | paused | budget_paused
  created_by, created_at, last_fired_at, next_fire_at
}
```

4.4 Budget (new)

```
Budget {
  scope      org | team | agent
  scope_id, period, cap_units, used_units, projected_units
  on_80pct   notify [ user_id ]
  on_100pct  pause scheduled + event triggers // NEVER on_demand
            notify [ maintainer, super_admin ] // never silent
  rollover   one_period
}
```

4.5 AgentProposal and SelfModPolicy (new)

```
AgentProposal {
  id, agent_id
  kind      instruction | scenario | source | param
  title, body, evidence, confidence
  generated_from [ run_id | request_id | vote_id ]
  state        proposed | in_test | published | declined
  decided_by?, decided_at?
}

SelfModPolicy { // org default, per-agent override
  may_propose_instruction  default true
  may_propose_scenario     default true
  may_propose_source       default true
  may_self_edit            default FALSE
  may_create_triggers      default FALSE
  may_clone_subagents      default FALSE
}
```

4.6 Unchanged from the prototype

`Installation`, `Outcome`, `InputParam`, `Team`, `BusinessContext`, `Request`, `Scenario` keep their prototype definitions, with two additions:

`Installation` gains `pinned_version?` and `pin_expires_at?` (§7.8), and `scope_grants` [`{ source, granted: bool, granted_at }`] (§7.5).

5 · Flow A — Creating an agent

Actor: team admin. **Goal:** from intent to a published agent the team can install. **Budget:** under 20 minutes for a first agent.

The 20 minutes is not an aspiration. **[msr]** The competitor's measured time from clicking Create to a running scheduled agent with a working first run was **17 minutes 30 seconds**, and roughly four of those were the operator's own tooling mistakes. A product whose authoring loop is slower than that has no claim on this market. BR-C-036 makes it an acceptance criterion.

5.1 A1 · Create

Screen: `#/a/new`

BR-C-001 **MUST** Clicking *New agent* creates a persisted draft immediately and routes to its workspace. No modal, no name prompt, no required field before the object exists. **[obs]**

BR-C-002 **MUST** The draft has a generated name the author can change at any time and is never asked for. The author-facing label reads *Untitled agent — name it whenever*.

BR-C-003 **MUST** A draft is private to its author: no installs, no store listing, no triggers armed, no cost.

BR-C-004 **MUST** The workspace shows exactly four next steps, phrased in the author's own voice rather than the platform's, each linking to the surface that satisfies it:

STEP	THE VOICE	OBJECT
Tell me when to run	"I want you to start working without anyone asking."	Trigger
Give me the team's context	"Everything you'd tell a new joiner in their first week."	BusinessContext
Decide what your team can change	"Which knobs are safe for someone who isn't you?"	input_schema
Say what good looks like	"One number this should move."	Outcome

[obs] This is the teardown's single most-liked pattern, recorded as "steal this pattern verbatim": a fresh agent workspace that suggests *Set up a trigger — I want you to start doing things without me having to ask*, *Build a live dashboard*, *Create a skill — I want to teach you how I do things*, *Personalize yourself*. It teaches the object model while selling proactivity. The four steps above are the same device pointed at Bench's objects.

BR-C-005 **SHOULD** The screen offers four starting points: describe it in chat (default), fork a published agent, paste an SOP, or walk a top performer's week.

BR-C-006 **MUST** Drafts untouched for 72 hours appear under *Your unfinished drafts* here and on the team pulse, with the author, the age and what is missing. **[obs]** A deliberately half-finished agent drew no nudge anywhere outside the workspace its author had already left: "the product notices abandonment only inside the workspace you already left, which is precisely where you will not be."

5.2 A2 · Describe

Screen: `#/a/build`, tab *Describe*

BR-C-007 **MUST** The chat interviews the author and produces a complete draft config. The author never faces an empty form. This is the prototype's settled decision and is unchanged.

BR-C-008 **MUST** Every field the chat sets is editable in the Config tab, and editing a field does not lock it against later chat edits.

BR-C-009 **SHOULD** The sidebar shows which of the nine config fields are filled, live.

[cmp] Chat-first authoring with a deterministic escape hatch, and never a canvas as the front door, is the strongest convergent signal in the comparison: three well-funded practitioners exited visual workflow authoring for the mass buyer within a year of each other. The prototype's decision already agrees; this is confirmation, not a change.

5.3 A3 · Configure

Screen: `#/a/build`, tab *Config*

BR-C-010 **MUST** Configuration is one surface with named sections — Identity, When it runs, What it may read and write, Instructions, Autonomy and guardrails. There is **no tab called "Settings"**.

[obs] In the competitor, configuration splits across a tab called *Settings* (name, description, secrets, danger zone) and one called *Agent* (instructions, triggers, connectors, skills, knowledge, sub-agents, abilities). The journal: "I looked for instructions under Settings first and found a dead end. The tab that matters most is named the least descriptively."

BR-C-011 **MUST** Each declared scope states, in one plain sentence, what it lets the agent see — text the installer will read at consent time (§7.5), so it is authored once, here.

BR-C-012 **MUST** Scopes are marked `required` or optional. A required scope declined at install blocks the install with an explanation; an optional one declined installs a degraded agent that says what it cannot do.

BR-C-013 **MUST** Instructions inherit the team's business context automatically, and the screen says so and links to it.

5.4 A4 · The trigger, in a sentence

Screen: `#/a/build`, tab *Config*, section *When it runs*

BR-C-014 **MUST** The schedule is authored as **natural language**. The author types "Every weekday at 8:15" or "Every day at 7, and when an account crosses T-90". There is no cron syntax anywhere in the product.

BR-C-015 **MUST** The parse is **echoed back before save**, in a labelled block, in the product's words: *What I understood: Every day at 07:00 IST, plus whenever an account crosses T-90. First run tomorrow 07:00.*

BR-C-016 **SHOULD** Five suggestion chips cover the common cadences.

[obs] "The scheduled form takes 'Every 30 minutes' as typed text or a chip, has a Generate button that parses it, and shows the parsed result. No cron syntax anywhere. **This is the best trigger UX I have used.**" It is the one interaction in the teardown scored as category-leading, and it is copied deliberately.

BR-C-017 **MUST** Directly beneath the parse, and before the author can save, the screen states the **projection**: runs per day, runs per month, cost band, projected units, and the team's remaining headroom this month.

[msr] This requirement exists because of what sits immediately next to that excellent trigger builder in the competitor: nothing about cost. Measured per triggered run on a real account — a heartbeat agent replying one word, **7 credits**; a run that only blocks, **19**; a 30-row scrape, **33**; a modest monitoring agent, **37–85** over four cycles. At the cadence that builder itself suggested, one agent burns a 20,000-credit monthly allowance in **5 to 11 days**. Four agents on 15-minute schedules did it in **under two**. A do-nothing agent on a 15-minute schedule does it in **thirty**. Nothing on the screen said so. The counterpoint that shapes the whole design: the entire four-day program — every build, chat and manual run — cost about 1,200 credits. **The interactive product is cheap. The scheduler is the cliff.**

BR-C-018 **MUST** A projection exceeding the team's remaining headroom is a **warning, not a block** — with the number, and the two ways out (raise the cap, or trim the cadence).

5.5 A5 · Autonomy and guardrails

BR-C-019 **MUST** Three autonomy levels — Suggest, Draft, Auto-act — with the plain-language meaning shown against each. Unchanged from the prototype.

BR-C-020 **MUST** Auto-act requires a written reason, an owner, and a list of permitted actions. Only a super admin may grant it.

BR-C-021 **MUST** Each hard stop generates its own adversarial test scenarios, and those scenarios are not deletable.

BR-C-022 **MUST** One guardrail class is mandatory on every agent: **declared unknowns** — the agent names what it does not know rather than filling the gap.

[obs] This is the behaviour worth specifying rather than hoping for. Given deliberately ambiguous instructions, the competitor's agent stopped before acting: it had searched the chat, the workspace, its instruction file and prior conversations, found no URL and no tracker, and said exactly that — "I did not guess a competitor URL, did not write a new tracker, and did not touch email/Slack or any external site." Pointed at a bot-blocked page it tried three URL variants, reported the block, **refused to substitute third-party snippets**, and asked two specific questions. The teardown: "This is the honest-failure behavior the agent-platform market claims and rarely shows."

5.6 A6 · What users can change

Screen: `#/a/build`, tab *What users can change*

BR-C-023 **MUST** The author declares each exposed parameter with key, label, type, options, default, and — required — a one-line reason it is open.

BR-C-024 **MUST** The author writes the locked list in their own words: what cannot be changed, and why.

BR-C-025 **SHOULD** The screen previews the install form the member will see.

BR-C-026 **SHOULD** The builder surfaces parameters that nobody ever changes, from live data, and suggests locking them.

5.7 A7 · The outcome metric

Screen: `#/a/build`, tab *Outcome metric*

BR-C-027 **MUST** An `Outcome` is required to publish: metric name, unit, direction, baseline, target, source system, source query, attribution window.

BR-C-028 **MUST** The metric reads from a **source system**, not from platform run data. Self-reported time savings are not an available option.

BR-C-029 **SHOULD** The default attribution is installers versus non-installers, same team, same quarter, and that option becomes unavailable above ~85% adoption.

[cmp] No shipping platform in the 27-company comparison requires an outcome at publish. The nearest analogue is a publish gate on an evaluation pass rate. This is the prototype's strongest original idea and it is carried unchanged.

5.8 A8 · Test, and try it for real

Screens: `#/a/test`, and `#/a/build` tab *Try it for real*

BR-C-030 **MUST** Scenarios are generated from the config — sources, instructions, each hard stop — and regenerate whenever any of those change.

BR-C-031 **MUST** The vocabulary "eval", "grader", "dataset", "rubric" appears nowhere. The author gets: the situation, what should happen, what did happen, and what broke.

BR-C-032 **MUST** A failing **guardrail** scenario is a hard block on publish. A failing core or domain scenario is a warning.

BR-C-033 **MUST** A version that breaks a previously passing scenario cannot be published without an explicit override, and the override is logged with its author.

[voc] The reason to couple the gate to the tests at all: the competitor shipped a 17-operation evaluations API and exactly **1 of 6,377 forum posts** contains the word "evaluation" — and that one is noise. Its agent versions have no deploy or rollback endpoint, so an evaluation can detect a regression and nothing can act on it. An eval coupled to a gate is a control; an eval coupled to nothing is a report nobody reads.

BR-C-034 **MUST** The builder offers a **dry run**: the draft executes against real data with every write suppressed, and every suppressed write is listed as *would have*.

BR-C-035 **MUST** A dry run produces a full trace, readable at `#/a/run/<id>`, and is billed to the draft rather than the team's live budget.

[obs] The competitor's build loop ends in a real run at minute seventeen — sixteen steps, both target pages fetched, and a versioned artifact saved without being asked twice. Scenarios are synthetic; this is the moment the author learns whether the thing works. An author who publishes something they have never seen run hands first contact with reality to a teammate's Monday morning.

BR-C-036 **MUST** (*acceptance*) A team admin with a job in mind reaches a published agent in **under 20 minutes**, measured create-click to publish, on the pilot dataset.

5.9 A9 • Publish

Screen: `#/a/publish`

BR-C-037 **MUST** Publish is blocked on: no outcome metric; any failing guardrail scenario; an unapproved new scope; no budget cap. Everything else is a warning.

BR-C-038 **MUST** Publish shows the change summary — added, changed, removed — each line carrying who asked for it.

BR-C-039 **MUST** Publish shows the blast radius: people affected, runs per day, settings that will reset, and any new permission each person will be asked for.

BR-C-040 **MUST** Publish sets the agent's budget cap, defaulted from the projection the author saw at BR-C-017 .

BR-C-041 **MUST** Rollout is one of: everyone on the team; pre-install for people who have not got it; or a ring of three, then everyone after 48 hours.

BR-C-042 **MUST** A rollback to the previous version is available from the publish screen and from version history, moves every install, preserves every setting, and is logged.

5.10 Edge cases — creation

#	CASE	BEHAVIOUR
CE-1	Author leaves mid-draft	Draft persists, appears in unfinished drafts after 72h, and is reassignable by the team admin
CE-2	Two admins edit one draft	Last write wins per field, with a "changed by Meera 2 min ago" marker. No locking. [jdg]
CE-3	Natural-language schedule cannot be parsed	Parse echo says what it could not read and asks for a rephrase. Never silently picks a default.
CE-4	Projection exceeds the team cap	Warning with the number and two remedies. Not a block (BR-C-018).
CE-5	Dry run finds no data	Reports <i>nothing to do</i> and says which source was empty. Not an error.
CE-6	Outcome metric's source system is not connected	Publish blocked; the screen links to the connection request.
CE-7	Scenario generation fails	Publish blocked with the reason. An agent with no tests cannot go live.
CE-8	Guardrail scenario fails after a previously passing version	Hard block, plus the regression diff (BR-C-033).

6 · Flow B — Sharing an agent

Actors: team admin publishing, member installing, super admin governing.

6.1 The finding this flow exists to invert

This is the most important page in the document.

[obs] In the competitor, the team namespace mirrors the personal one completely — home, agents, knowledge, skills, artifacts, connectors, tasks, secrets — and every surface of it is empty. The operator set org-wide access on a live agent and verified it persisted. Then: the team's agent list did not contain the agent; the team's home showed no recent activity and no upcoming triggers, even though that agent's 30-minute schedule was firing; the team's knowledge base stayed empty. The agent's own settings tab has no move-to-team control at all — its affordances are *Make a Copy* and a *Danger Zone*. The only sharing path is a Google-Docs-shaped dialog: invite by email, or set access to Restricted / Organization / Anyone.

The teardown's verdict: **"Sharing is permission-only. Ownership, listing, trigger, knowledge and history all stay personal. The team namespace is a parallel silo you would rebuild in, not a destination you move to."**

Bench's entire thesis is that one person's expertise becomes the team's. A shared agent that still belongs to one person reproduces the individual-subscription problem inside the platform. So:

BR-S-001 **MUST** The first publish **transfers ownership**. `owner_team_id` moves from the author's drafts to the team; `maintainer_id` is set. There is no path back to personal ownership.

BR-S-002 **MUST** `author_id` is permanent and displayed — on the store card, in every release note, and in the analytics. Credit is the author's; custody is the team's.

BR-S-003 **MUST** Every live agent has exactly one maintainer. It defaults to the author, is reassignable by the team admin in one click, and is **forcibly reassigned** when the author's account is deactivated, with the super admin notified.

BR-S-004 **MUST** The maintainer receives: failed-run notifications, budget warnings, requests, and agent proposals.

This answers the prototype's own open question — *"Who owns an agent when its author leaves? Four people authored half of Plum's agents and the object has one `author_id`."*

6.2 B1 · Audience

Screen: `#/a/share`

BR-S-005 **MUST** Audience is one of: the owning team; everyone in the org; or named people outside the team.

BR-S-006 **MUST** Audience decides who may **install**. It never decides ownership, and the screen says so in those words.

[obs] The three-level shape is borrowed deliberately — Restricted / Organization / Anyone is familiar because it is the Google Docs model, and familiarity is worth having. It is the *semantics* underneath it that are inverted.

6.3 B2 · The store

Screen: `#/m/store`

BR-S-007 **MUST** A store card leads with what the agent does, when it runs, whether it can act on its own, and the metric it moves — with the baseline, the current value and a sparkline.

BR-S-008 **MUST** Member-facing surfaces carry no version numbers, no verified badges, and no autonomy jargon. The card says "Every weekday, 8:15" and "Nothing sends without you". Carried from v2, law L10.

BR-S-009 **SHOULD** The store shows install counts and the team average.

6.4 B3 · Preview before consent

Screen: install modal

BR-S-010 **MUST** Before any permission is requested, the install screen offers "**Show me mine**" — a read-only backfill of the agent against the installer's own data, rendered as the real output.

BR-S-011 **MUST** The backfill writes nothing, sends nothing, and is not billed to any budget.

BR-S-012 **MUST** A scheduled agent fires one **immediate backfill run** on install, so the first value does not wait for tomorrow's trigger.

[obs] The competitor's onboarding shows a drafted reply, a meeting brief, a live dashboard and a terminal session across five steps before a single connect button appears — permissions are asked at steps six to eight, after value has been established. The prototype's own flow document identifies the same problem from the other end — *"a scheduled agent installed at 4pm first speaks at 08:15 tomorrow ... without a backfill it feels dead for a day and install-day churn spikes"* — and marks it as a decision to make. This settles it.

6.5 B4 · Scope consent, one source at a time

BR-S-013 **MUST** Each declared source is a **separate grant** the installer can decline individually, with the plain-language sentence authored at BR-C-011 shown against it.

BR-S-014 **MUST** The default is least-privilege: only the sources the agent marks `required` are pre-granted. No "unrestricted access" option exists anywhere.

BR-S-015 **MUST** The agent runs with the **installer's own credentials**. It can never read a deal, a thread or a file the installer could not open themselves, and the install screen says so.

BR-S-016 **MUST** Declining an optional source installs a degraded agent; the agent states what it cannot do, in its output, on every affected run.

[obs] [voc] The competitor's scope modal pre-selects, labelled *Recommended*, "Unrestricted access to everything Gumloop can do with this connector", with "Control scopes" as the alternative. Its forum carries 28 oauth-permission topics, including a company refusing the Slack integration's scope list outright and an Office 365 admin grant being declined. Per-user OAuth wins activation and loses the security review; granular scopes are how you keep the first without the second. Plum's compliance function will read this one screen.

6.6 B5 · Install semantics

BR-S-017 **MUST** The following split is enforced and stated on the share screen:

Parameter values	copied	The installer's own, set from the author's defaults
Credentials	copied	The installer's own OAuth grant
Run history and threads	copied	Private to the installer
Instructions	referenced	The team's; changes when the maintainer publishes
Guardrails and hard stops	referenced	The team's; never overridable by an installer
Scopes	referenced	Declared by the agent, granted per source by the installer
Outcome metric	referenced	One definition for everyone
Version	pinned	Publishes advance it unless deliberately pinned

[obs] Specified explicitly because the competitor's equivalent is implicit and surprising: sharing grants chat access, config-edit rights and a private history, while ownership, listing, triggers and knowledge all stay with the author. Two engineers reading an underspecified screen build two different products.

6.7 B6 · Uninstall and requests

BR-S-018 **MUST** Uninstalling asks for a reason, from a short list plus free text. The reason is recorded on the `Installation` and appears in the maintainer's requests inbox.

BR-S-019 **MUST** Hitting a locked setting offers *Suggest a change*, which creates a `Request` addressed to the maintainer.

BR-S-020 **MUST** Requests carry vote counts and the uninstalls they explain.

BR-S-021 **MUST** A request shipped in a version is linked to that version, and the release note names the person who asked.

This is the adoption loop and it is a mechanism, not a nicety: members cannot fork, so the pressure has to go somewhere that produces a change.

6.8 B7 · Cross-team spread

BR-S-022 **MUST** A **team admin** may take another team's published agent as a template. The fork is a new agent, with a new owner, a new maintainer and a fresh outcome baseline.

BR-S-023 **MUST** Lineage is recorded on both ends and displayed on both: *forked from Sales · Renewal Defender v3.1*.

BR-S-024 **MUST** The origin's maintainer is notified when their agent is forked.

BR-S-025 **SHOULD** The share screen shows each fork's drift in one line.

[cmp] Without a one-to-many channel, distribution stays one-to-one and a good agent spreads by being rebuilt by hand. The comparison's recommendation on this — pair versioned, shareable output with a gallery so distribution is one-to-many — applies to agents as much as to their artifacts.

6.9 B8 · The deprecation contract

BR-S-026 **MUST** Removing a setting, a source or a section starts a contract:

1. **Notice** — 14 days, in the feed and by DM, naming what goes and why.
2. **Automatic migration** — parameter values are migrated, never silently dropped. A removed key is retired and stated; a new key takes its default and is stated.
3. **Pinning** — anyone may pin to their current version for up to 60 days. Pinned installs are listed for the maintainer.
4. **No paywalled history** — runs, traces and history stay readable regardless of billing state.

[obs] [voc] Each clause answers a documented failure. Model deprecations broke running agents with no notice and the staff remedy was customer labour: *"We deprecated GPT 4o-mini for GPT 4.1-mini, if you delete and add back the Ask AI and the Extract Data nodes you should be all set."* Loop modes, JavaScript support and templates were removed. A free-tier sunset locked non-payers out of their own chat history: *"I get continuously redirected to /upgrade and can't reach my chats at all."*

BR-S-027 **MUST** Four or more pinned installs on one agent raises a flag to the maintainer: the last publish was wrong.

6.10 Edge cases — sharing

#	CASE	BEHAVIOUR
SE-1	Installer declines a required scope	Install blocked, with which source and what it is for. Offers to request the grant from the super admin.
SE-2	Installer's token expires	Next run is <code>failed</code> with the cause named; the installer is asked to reconnect; the maintainer is not paged for one person's token.
SE-3	Maintainer leaves the company	Ownership stays with the team; maintainer forcibly reassigned to the team admin; super admin notified.
SE-4	Author leaves the company	Nothing moves. <code>author_id</code> is a permanent credit on an object the team owns.
SE-5	An agent is forked and the origin is retired	The fork is unaffected. Its lineage shows the origin as retired.
SE-6	Publish while someone is mid-install	The install completes on the version it started with, then advances.
SE-7	Pin expires	Install auto-advances; the installer is told, with the diff since their pinned version.
SE-8	Org-wide audience on a team-owned agent	Allowed. The team still owns and maintains it; the super admin is notified.

7 · Flow C — Monitoring

Actors: member watching their own agents, team admin watching the team's, super admin watching the org's.

This is the largest gap between the prototype and a product, and it is also where the teardown's findings are densest. Everything the teardown found broken in a shipping product lives in this flow.

7.1 C1 · Run status

BR-M-001 **MUST** Every run ends in exactly one of five statuses:

STATUS	MEANING	MEMBER SEES
completed	Did the job and produced something	(the output)
nothing_to_do	Ran, found nothing worth the person's attention	"Nothing today"
blocked	Stopped and asked a question	"Needs you"
failed	Could not finish; cause named	(nothing — the maintainer is told)
partial	Did some of the work and named what it skipped	(the output, with the gap stated)

BR-M-002 **MUST** A run that produced nothing is **never** completed . A run that stopped for a missing input is **never** completed . A run that hit a hard block is **never** completed .

[obs] This requirement exists because of one observation, and it is the sharpest in the teardown. Two runs that could not do their job — one stopped for a missing input, one blocked by the target site — both landed in the ledger as **Completed**, the second at 25 credits. After seven scheduled runs including a blocked one, the *Failed* tab read "No history yet" and the *Approval Required* tab read "No history yet" — even though the agent had asked direct questions mid-run. The teardown: **"A scheduled agent that blocks on a question has no visible surface outside its own chat: if you are not watching, you do not know it is waiting for you."**

BR-M-003 **MUST** Three consecutive `nothing_to_do` runs for one installer raise a **setup flag** on the team pulse, naming the parameter most likely responsible.

A digest that finds nothing for three days is usually not a quiet week. In the pilot data Farhan's floor is set to ₹10 lakh and his book is SMB — a configuration problem wearing the costume of a quiet week, and indistinguishable from one in every incumbent's analytics.

7.2 C2 · When a run stops: the Ask

BR-M-004 **MUST** A run that cannot finish sets `status: blocked` and writes an `Ask` : the question, why it did not decide for itself, and two or three options.

BR-M-005 **MUST** An `Ask` raises an item on the installer's **Waiting on you** surface (`#/m/waiting`) and a row in the team admin's run log. It **MUST NOT** exist only inside a thread.

BR-M-006 **MUST** If an `Ask` is unanswered when the agent would next run, both the installer and the maintainer are notified.

BR-M-007 **MUST** Answering an `Ask` **resumes** the run from the step it stopped on. It does not restart it. A restart re-reads every source and bills twice.

BR-M-008 **MUST** The *Waiting on you* surface carries no run ids, costs, step counts or status vocabulary. It carries the question, why the agent did not decide, and the options. Law L10.

[cmp] The resumability requirement is borrowed from the one platform that models this properly: sessions pause in an `approval_required` state, a single call resolves up to twenty pending asks and returns a cursor that resumes the event stream. The mechanism is right; what is missing there is any surface that tells a person the ask exists.

7.3 C3 · When a run fails

BR-M-009 **MUST** A `failed` run records a `Failure` with three fields: the **cause**, what the agent **tried**, and a **suggested fix**.

BR-M-010 **MUST** The maintainer is notified immediately, with the cause in the notification body.

BR-M-011 **MUST** The suggested fix links to the screen that applies it.

BR-M-012 **MUST** An agent that cannot complete **MUST NOT** substitute a plausible alternative. Falling back to a default list, estimating from partial data, or reading numbers off an unparseable document are failures, not degraded successes.

[obs] The behaviour to preserve, observed and worth quoting: pointed at a blocked page, the competitor's agent tried three URL variants, reported *"LinkedIn is blocking the fetch, so I do not have a live follower count"*, explicitly refused to substitute third-party snippets, and asked two specific questions. That is the best failure reporting in the category. The product's own mistake was to record it as a success.

7.4 C4 · The run log

Screen: `#/a/runs`

BR-M-013 **MUST** The run log lists every run of every agent the viewer is entitled to see, newest first, filterable by status.

BR-M-014 **MUST** Each row carries: agent, run id, a **one-line outcome**, trigger reason, who it ran for, start time, duration, status and cost band.

BR-M-015 **MUST** The one-line outcome states what the run actually did — *"3 of 4 quotes normalised · Star Health PDF is a scan, named and skipped"* — not a step count.

[obs] Each triggered run in the competitor spawns its own chat thread and the ledger records only cost and status. "If you are not watching the threads, you do not know what your agents did — the ledger will not tell you."

BR-M-016 **MUST** A row opens the trace.

BR-M-017 **SHOULD** The log's header tiles show: runs today, runs needing a human, `nothing_to_do` count, and spend today against the month.

[voc] The reason this screen is P0 and not P2: **"Could really use a log"** is the highest-engagement organic feature request on the competitor's own forum, ahead of agent customization and connector depth. Debuggability carries 65 tagged complaint topics, and reliability is the single largest complaint family at 53% of all get-help topics — every reliability incident is lengthened by the absence of a log.

7.5 C5 · The trace

Screen: `#/a/run/<id>`

BR-M-018 **MUST** The trace lists every step with a label, a detail line, a timestamp, a duration and a kind.

BR-M-019 **MUST** Step kinds are `ok`, `warn`, `fail`, `auto`, `block`, and `warn` is used for a decision not to act — a missing source, a declared unknown, a refusal, a suppressed write.

BR-M-020 **MUST** The trace states what the run **read**, what it **wrote**, and what it **sent outside the organisation** — the last explicitly, even when the answer is "nothing".

BR-M-021 **MUST** An `auto` step — an agent acting without a human — is visually distinct and appears in the audit log.

BR-M-022 **SHOULD** The trace shows the run's cost and the agent's month-to-date position against its cap.

[cmp] This screen is the answer to the one thing a switcher gives up: *"A run here is a chat thread with '16 Steps' you can read but not open; a Zapier or n8n run is an inspectable graph you can debug."* The `warn` steps are the part nobody ships, and they are the most useful rows on the screen.

7.6 C6 · Schedules

Screen: `#/a/schedules`

BR-M-023 **MUST** One screen lists every trigger that can start a run without a person asking, across the team: agent, label, the author's own words, cadence, next fire, runs per month, projected units, and state.

BR-M-024 **MUST** **Pause** and **Stop** are labelled buttons on the row, keyboard reachable, with accessible names. Not inside a menu. Not an icon alone.

BR-M-025 **MUST** Stop shows what stops and what is kept — the agent, every install, every setting and every past run survive; only the clock stops.

BR-M-026 **SHOULD** Pause is offered as the lighter alternative in the stop dialog.

[obs] "Creation is one visible click; deletion took six passes." The delete affordance was an unlabelled ellipsis inside a drawer with **no aria-label** — invisible to text-based navigation and to screen readers — while every labelled menu on the page opened something else. Finding it cost roughly 640 credits of accidental runs.

BR-M-024 is the whole finding turned into a requirement.

7.7 C7 · Budgets

BR-M-027 **MUST** Budgets exist at three scopes: org, team and agent. A team cannot raise its own cap. There is no per-person cap.

[jdg] People are not the unit that runs away; schedules are. A per-person cap punishes the heaviest user, who is usually the one getting the most value.

BR-M-028 **MUST** At 80% of any cap the owner of that scope is notified, with the projection and the largest contributor named.

BR-M-029 **MUST** At 100% of a cap, **scheduled and event triggers pause**. On-demand runs continue.

[msr] Because the cliff is the scheduler. The same four-day program that burned 11,300 credits a day on a four-agent schedule grid cost about 1,200 credits in total for every build, chat and manual run across four days. A member typing a question must never hit a wall that a schedule created.

BR-M-030 **MUST** A budget pause notifies the maintainer and the super admin, with the number and the cause. A silent auto-disable is prohibited.

[voc] Observed in the competitor's forum as exactly that failure: triggers auto-disabling after credit exhaustion and silently missing form submissions.

BR-M-031 **MUST** Unused allowance rolls over for one period.

BR-M-032 **MUST** Caps, projections, rollover and the spend screen ship on **every plan**.

[voc] The anti-pattern being avoided, with its documented cost: the competitor built per-role monthly caps correctly — audit-trailed, API-manageable, highest-limit-wins across roles — then gated them, along with rollover and spend insights, behind its top tier. Five of nine documented churn stories on its own forum are billing mechanics; none cite the subscription price. The cure was priced above the pain.

7.8 C8 · The cost unit

BR-M-033 **MUST** The meter is the **run**, in three published bands — light, standard, heavy. There is no second currency and no per-token arithmetic on any user-facing surface.

BR-M-034 **MUST** Members never see a cost figure. Admins always do.

BR-M-035 **MUST** Every surface showing cost reads the same ledger (law L4).

BR-M-036 **MUST** Nothing bills that does not appear as a run (law L7). Background classification, indexing and re-sync each appear as runs or do not bill.

[cmp] The trust winners among meters are the coarse ones — executions, or one credit per run. The most transparent fine-grained meter in the comparison — published unit price, published orchestration percentage, published per-minute compute — coexists with the category's worst bill-shock complaints: *"it spent 60000 credits (in about 4 minutes)"*, *"it cost me 900 credits"* for one search. Transparency is necessary and not sufficient. The deliverable is predictability.

7.9 C9 · Adoption and outcome

Screen: `#/a/analytics/<id>`

BR-M-037 **MUST** The agent analytics screen joins **usage** to **outcome** in one per-person table: runs, items acted on, that person's metric value, and their segment.

BR-M-038 **MUST** acted , dismissed and ignored are tracked separately.

BR-M-039 **MUST** ignored is computed at the agent's next run — 72 hours for on-demand agents — and the window is stated on screen next to the number.

Dismissed is a judgement call; ignored is a design failure. They look identical in every incumbent's analytics and they are not the same fact. The prototype defines the distinction and leaves the window undefined, which makes it unbuildable; this fixes it.

BR-M-040 **MUST** Individual usage rankings are visible to admins only and are never published to the team.

BR-M-041 **MUST** The drop-off panel names people: who uninstalled and why, who installed and went quiet, who opened the store and never installed.

[cmp] The nearest thing shipping is a per-user × per-agent table that counts responses received — a volume proxy with no outcome attached. Joining "how much did you use it" to "did your number move" is the differentiated view.

7.10 C10 • The pulse

Screen: `#/a/pulse`

BR-M-042 **MUST** The team pulse opens with a **Needs you today** strip listing, in priority order: failed runs, blocked runs, budgets about to bite, quiet installs, and drafts that have stalled. Each row states the cause and links to the fix.

BR-M-043 **MUST** An empty strip reads *nothing needs you today* explicitly. Silence is never left to be interpreted.

[obs] The part the competitor gets right and is worth copying: recent activity, upcoming triggers and most-used agents on the landing surface make the retention loop visible — "the product pulls you back to work it did without you." What it does not do is tell anyone when something needs a person. Every row in BR-M-042 would be invisible there.

7.11 Edge cases — monitoring

#	CASE	BEHAVIOUR
ME-1	A run blocks and the installer is on leave	Escalates to the maintainer at the next trigger. Out-of-office suppresses ignored scoring.
ME-2	Same Ask raised on consecutive runs	Deduplicated into one item with a count. The agent does not re-ask daily.
ME-3	A run fails because a source is down for everyone	Deduplicated to one maintainer notification naming the source and the count of affected runs.
ME-4	Budget cap hit mid-run	The in-flight run completes. The next scheduled run does not start.
ME-5	Cap raised after a pause	Schedules resume at their next natural fire, not immediately.
ME-6	An agent auto-acts and the action was wrong	The auto step is in the trace and the audit log with its ceiling check; the member can report it, which creates a request against the agent.
ME-7	Trace exceeds the size limit	Steps are kept; step details are truncated oldest-first, and the truncation is stated.
ME-8	A trace is needed after 30 days	Pin a run to exempt its trace from expiry. Pinning is available from the run row.
ME-9	Two agents claim the same outcome metric	Allowed. Attribution splits by which agent's item the person acted on.

8 · Cross-cutting requirements

8.1 Notifications

BR-X-004 **MUST** Six events produce a notification, and no others produce one by default:

EVENT	WHO	WHEN
A run failed	Maintainer	Immediately
A blocked run is still unanswered at the next trigger	Installer + maintainer	At the next trigger
A budget reaches 80% of a cap	Scope owner	Once per period
A budget pauses schedules	Maintainer + super admin	Immediately
An agent you installed was published with a breaking change	Installer	At publish, and 14 days before a removal
Your request shipped	Requester	At publish

BR-X-005 **MUST** Slack is a first-class destination, not a mirror: approvals and blocked runs are **actionable** in Slack.

[cmp] The recommendation this follows, verbatim from the platform comparison: make the approval surface the place where the approver already is.

BR-X-006 **MUST** Notifications are digestible — more than three of the same class within an hour collapse into one.

BR-X-007 **SHOULD** Every notification class can be turned down per person, except failure and budget-pause for a maintainer.

8.2 Approvals

BR-X-008 **MUST** Approval requirements are expressed as **rules conditioned on the content of the action**, not only on the agent's autonomy level. The three rules that ship by default:

1. Anything addressed to a recipient outside the organisation
2. Anything stating a monetary figure above a threshold the admin sets
3. Anything touching an account on a named list

BR-X-009 **MUST** An admin may allow drafts matching no rule to be released without approval, per agent.

BR-X-010 **MUST** Up to 20 items can be approved in one action.

BR-X-011 **MUST** The approval queue and the *Waiting on you* queue are **separate surfaces**. An approval is "should this leave the building"; a blocked run is "the agent needs a decision to continue".

[obs] They are conflated in the competitor and the result is that neither fires: the approval queue is for outbound actions, agent questions never reach it, and the queue read "No history yet" for the whole program.

8.3 Credentials and scopes

BR-X-012 **MUST** Agents run as the installing user, through per-user OAuth. An agent can never read what its installer could not open themselves.

BR-X-013 **MUST** Scope grants are per source, per user, least privilege, revocable, and listed on a screen the user can find.

BR-X-014 **MUST** A revoked or expired grant produces a `failed` run naming the source and asking that user to reconnect. One person's expired token never pages the maintainer.

BR-X-015 **MUST** New scopes introduced by a publish are consented to **individually by each installer**. They are never inherited from the author.

This is the prototype's "biggest fork" — per-user OAuth versus a service account — settled in favour of per-user, which is what the compliance function at a regulated broker will accept. The cost is real and accepted: a consent screen per install, and scheduled agents that break when a token expires. BR-X-014 is the mitigation.

8.4 Audit

BR-X-016 **MUST** The audit log records: publishes, rollbacks, ownership and maintainer changes, scope grants and denials, auto-acts, budget-cap changes, budget pauses, self-modification policy changes, proposal decisions, uninstalls with reason, and overrides of a failing test.

BR-X-017 **MUST** Every audit entry names a person or an agent, never "system".

BR-X-018 **MUST** The log is exportable.

8.5 Business context

BR-X-019 **MUST** A team's business context — how it works, targets, incentives, what separates a good performer from a weak one, its calendar, its glossary — is a first-class object, attached automatically to that team's agents.

BR-X-020 **MUST** Team setup enforces its order: context before the first agent.

BR-X-021 **MUST** The context interview is conversational and the platform asks the questions. A blank form is not acceptable.

[obs] Confirmed from the other direction: the competitor's knowledge base accepts content only by sync or upload — *"For a product called Brain, you cannot think into it directly."* Bench's context is typed, and that is the right call.

8.6 Self-modification

BR-X-022 **MUST** An agent may **propose** a change to its own instructions, scenarios, sources or parameters. A proposal enters the normal draft → test → publish pipeline as a version authored by the agent.

BR-X-023 **MUST** No proposal takes effect without a human publish.

BR-X-024 **MUST** A proposal carries its **evidence**: the runs, requests or votes it came from, and a confidence.

BR-X-025 **MUST** A proposed **new source** cannot be approved by a team admin; it goes to the super admin with the run count that justifies it.

BR-X-026 **MUST** The self-modification policy is org-level, super-admin-only, audited, and defaults to propose-only.

[obs] The ceiling being deliberately declined: that product ships *Allow Self-Updates*, *Triggers: AI Managed ON*, *Connectors: AI Discovery ON*, *Skills: AI Skill Editing ON*, self-cloning sub-agents, and tool discovery on AUTO — four AI-managed subsystems on one panel. **[cmp]** And the caveat that keeps this small: demand for learned skills is unproven. Roughly 3 of 708 harvested forum topics mention "skill" and none discuss the feature; the skills page in the teardown never moved off its two defaults across the whole program. This is supply-side differentiation, not observed demand. Ship it small.

8.7 Accessibility

BR-X-027 **MUST** Every interactive control has an accessible name. Icon-only controls carry `aria-label`.

BR-X-028 **MUST** No destructive or state-changing action is reachable only through an unlabelled control.

BR-X-029 **MUST** Status is never carried by colour alone — every status chip ships an icon and a word.

BR-X-030 **MUST** Text meets 4.5:1 against its own surface in both themes.

BR-X-027 and BR-X-028 exist because of one specific control: an unlabelled ellipsis with no `aria-label`, inside a drawer, as the only route to deleting a trigger.

8.8 Mobile

BR-X-031 **MUST** Three surfaces are designed for phone width: *Today*, *Approvals*, and *Waiting on you*. The rest must remain usable at 400px without horizontal scrolling of the page body.

9 • Non-functional requirements

BR-N-001 **MUST** A run's trace is visible while the run is in progress, streaming as steps complete. A user watching a run must see progress, not a spinner.

[obs] The one thing that made a two-minute wait tolerable: "the response streamed visibly and the steps counter ticked while it worked, so the wait felt like progress."

BR-N-002 **MUST** An on-demand run acknowledges within 2 seconds and shows its first step within 5.

BR-N-003 **SHOULD** A scheduled run fires within 60 seconds of its scheduled time. Drift beyond 5 minutes is a failure and is reported as one.

BR-N-004 **MUST** Concurrent triggers do not collide. Multiple agents firing on the same minute is a normal condition.

[msr] Confirmed as achievable: seven triggered runs across four agents on two colliding grid waves, all completed, no queue events, no failures.

BR-N-005 **MUST** Retention: runs 90 days; traces 30 days; outcome metric series indefinitely; audit log per the org's data-retention setting. A pinned run's trace is exempt.

BR-N-006 **MUST** Data survives a downgrade. Access to history is never contingent on billing state.

BR-N-007 **MUST** Run content is encrypted at rest and is never used to train a shared model.

BR-N-008 **SHOULD** The deployment ladder — cloud with a regional default, managed VPC, air-gapped — is a roadmap commitment for the regulated segment. Not drawn in the prototype; named here so it is not discovered late.

BR-N-009 **MUST** An agent's outcome metric query runs against the source system on a schedule the platform sets, not on every page load.

10 • Anti-requirements

Things Bench will deliberately not build, each with the evidence for the decision. These are as load-bearing as the requirements.

No canvas. Not as the front door, not as a power-user mode in v1. **[cmp]** Three well-funded practitioners exited visual workflow authoring for the mass buyer within a year of each other; one of them killed its own canvas eight months after shipping it. The canvas survives as a developer surface and as a governance surface, and dies as the mass-market authoring front door. Chat drafts, the form refines.

No member forking. Every route by which a member could keep a private variant is closed. Pressure goes into requests.

No second currency. One unit — the run — in three bands. No credits, no tokens, no per-minute compute line, no orchestration percentage. **[cmp]** The most transparent fine-grained meter in the comparison produces the category's worst bill-shock complaints.

No per-person spend cap. Schedules run away; people do not.

No connector-count race. Depth on the systems Plum actually uses, with a published action catalogue per connector, beats breadth. **[voc]** Connector depth is the third-largest complaint family on the competitor's forum at 226 topics — 91 of them specifically missing actions on connectors the vendor already markets.

No public AI-usage leaderboard. Individual rankings stay admin-only. Public leaderboards produce gaming and resentment, not adoption.

No "hours saved" metric. Not offered as an outcome option. Self-reported time savings do not survive contact with a P&L.

No silent anything. No silent deprecation, no silent trigger disable, no silent background meter, no silent success on a run that did nothing.

No agent that edits itself without review. §8.6.

No governance behind a paywall. Caps, rollover, the spend screen, the audit log and the run log ship on every plan. **[voc]** Pricing the cure above the pain is a documented churn mechanism; deleting the self-serve ladder to force enterprise ended, in one named case, in an acqui-hire.

11 · Open decisions

Each of these changes the product or the business. None should be settled by pointing at what a competitor does — and one caveat applies to all of them: that competitor's own commercial scale is unsourced. No ARR, no renewal, no NRR and no cohort data exists anywhere in the corpus assembled about it. The market position it occupies is verified unoccupied and its profitability is unproven.

1 · What is a licence metered on? Per seat is legible. Per run is fair. Per outcome is the most aligned and the hardest to contract. This document assumes **per seat plus metered runs** and specifies the meter accordingly. *Decision owner: commercial. Blocks: the plan page, the invoice, §7.8's bands.*

2 · Replacing individual AI seats, or layering on top? The org screen argues replacement. The top performers will keep their own seats to build with, which makes the real story "10 builder seats plus 78 platform licences" — a better story and a different price card. *Blocks: positioning, the cost comparison screen.*

3 · Whose logo is on this — Sarvam's or the customer's? Decides whether a store exists across customers (one broker installs an agent another broker built) or strictly per-tenant. Cross-tenant is a much larger moat and a much harder sell in insurance. *Blocks: the store's information architecture, §6.8's lineage model.*

4 · Is the outcome metric computed by the platform or read from the source system? Specified here as **source-system**, which is the harder build and the credible one. Computing from platform run data is easy and unconvincing. Reading from HubSpot, Gmail and policy admin means a connector per metric. *Blocks: the metric pipeline, §5.7.*

5 · Does a run bundle its retries? Drawn here as **yes** — a retry is part of the run that needed it. The alternative bills a failing agent more than a working one, which is the wrong incentive in both directions. *Blocks: §7.8's bands.*

6 · Sales or Account Management as the first team? AM's metrics are already numbers in a system — endorsement TAT, CD balance, renewal retention. Sales has the better story and the more visible top performer. AM is the easier pilot; Sales is the better case study.

7 · Does the pilot need WhatsApp on day one? Two of the three dormant users in the prototype's own data blame WhatsApp. In Indian B2B sales this may not be optional.

8 · Who at the customer actually authors agents? Everything here assumes a top performer who *wants* to build. If that person does not exist, the platform needs a services motion for the first five agents — and that is a different company.

9 · Should a member be able to pin a version indefinitely? Specified as 60 days. Longer fragments the team's practice, which is the thing the thesis is against; shorter removes the escape hatch from a bad publish.

12 · Acceptance criteria

The tests that decide whether this is built. Each maps to requirements above.

12.1 Creation

#	CRITERION	REQUIREMENTS
AC-C1	A team admin who has never used Bench reaches a published agent in under 20 minutes on the pilot dataset	BR-C-036
AC-C2	Clicking New agent produces a persisted, addressable draft in under 2 seconds with no modal	BR-C-001
AC-C3	"Every weekday at 8:15" parses, echoes back, and shows a monthly projection before Save is available	BR-C-014 – 017
AC-C4	Publish is refused with no outcome metric, with a failing guardrail scenario, with an ungranted new scope, and with no budget cap — each with its own message	BR-C-037
AC-C5	A dry run against real data writes nothing, lists every suppressed write, and produces a readable trace	BR-C-034 – 035
AC-C6	A version that breaks a previously passing scenario cannot publish without a logged override	BR-C-033
AC-C7	The word "eval", "grader", "dataset" or "rubric" appears nowhere in the product	BR-C-031

12.2 Sharing

#	CRITERION	REQUIREMENTS
AC-S1	After first publish, the agent is owned by the team, has a maintainer, and no control anywhere returns it to personal ownership	BR-S-001 – 003
AC-S2	Deactivating the author's account leaves the agent owned by the team with a reassigned maintainer and a notified super admin	BR-S-003 , SE-4
AC-S3	The install screen shows real output from the installer's own data before any permission is requested	BR-S-010
AC-S4	Each source can be declined individually; a declined optional source produces an agent that says what it cannot do	BR-S-013 , BR-S-016
AC-S5	A scheduled agent installed at 16:00 produces output within a minute, not the following morning	BR-S-012
AC-S6	A member has no route to fork	BR-X-002
AC-S7	Removing a setting produces 14 days' notice, migrates values, and offers a pin	BR-S-026
AC-S8	A fork records lineage on both ends and notifies the origin's maintainer	BR-S-022 – 024

12.3 Monitoring

#	CRITERION	REQUIREMENTS
AC-M1	A run that stops for a missing input is <code>blocked</code> , never <code>completed</code>	BR-M-002
AC-M2	A blocked run appears on the installer's <i>Waiting on you</i> within one minute and in the admin's run log	BR-M-005
AC-M3	Answering resumes the run from the step it stopped on, and the trace shows one run, not two	BR-M-007
AC-M4	An unanswered block escalates at the agent's next trigger to both installer and maintainer	BR-M-006
AC-M5	A failed run notifies the maintainer within one minute with the cause in the body	BR-M-010
AC-M6	Every trigger in the org is pausable and stoppable from one screen, from a labelled, keyboard-reachable control	BR-M-023 – 024
AC-M7	Crossing a budget cap pauses schedules, leaves on-demand working, and notifies two named people	BR-M-029 – 030
AC-M8	The cost of a run is identical on the run row, the agent page, the schedules screen and the spend screen	BR-M-035
AC-M9	Nothing appears on an invoice that does not correspond to a run in the log	BR-M-036
AC-M10	Three consecutive <code>nothing_to_do</code> runs raise a setup flag naming a parameter	BR-M-003
AC-M11	No cost figure appears on any member surface	BR-M-034
AC-M12	<code>ignored</code> is computed at the next run and the window is stated on screen	BR-M-039

12.4 The pilot test

AC-P1 One team, eight agents, fourteen people, four weeks. At the end:

- every agent has an outcome metric with four weeks of data against a baseline
- every failed run in the period has a named cause and a recorded resolution
- no run in the period is recorded as `completed` without having produced something
- the month's spend is within 10% of the projection shown when the last trigger was armed
- no person discovered a cost they had not been shown in advance
- the team admin can answer, from one screen, "what did our agents do yesterday?"

Appendix A · Evidence table

Every finding used, what it applies to, and the requirement it produced. Source: the founder teardown at `~/Desktop/code/gumloop` — `output/07-FOUNDER-TEARDOWN.md` , `research/gumloop/founder-journal.md` , `output/03-AGENT-BUILDING-INSIGHTS.md` , `research/gumloop/08-walkthrough.md` . Full digest in `work/01-gumloop-digest.md` .

#	FINDING	TYPE	VERDICT	REQUIREMENTS
A1	Create makes a working agent in ~12 seconds, no wizard	obs	Steal	BR-C-001 – 003
A2	The empty workspace nudges four next steps in the product's voice	obs	Steal	BR-C-004
A3	Natural-language cron, parsed and echoed; "the best trigger UX I have used"	obs	Steal	BR-C-014 – 016
A4	Time from create-click to a running scheduled agent: 17m 30s	msr	The bar	BR-C-036
A5	Config split across a <i>Settings</i> tab and an <i>Agent</i> tab; instructions in the wrong one	obs	Avoid	BR-C-010
A6	Self-updates, AI-managed triggers/connectors/skills, self-cloning sub-agents, all on	obs	Invert	BR-X-022 – 026
A7	The agent refuses to guess and names exactly what it did not do	obs	Specify	BR-C-022 , BR-M-012
A8	An abandoned agent draws no nudge outside its own workspace	obs	Invert	BR-C-006
A9	Knowledge enters only by sync or upload; you cannot type into it	obs	Invert	BR-X-021
B1	Nothing crosses personal → team; no move-to-team control exists	obs	Invert	BR-S-001 – 004
B2	Share dialog is Restricted / Organization / Anyone, plus invite	obs	Steal (shape)	BR-S-005 – 006
B3	Sharing grants chat, config-edit and a private history; ownership stays personal	obs	Specify	BR-S-017
B4	Shared artifacts run on the viewer's own credentials	cmp	Confirm	BR-X-012
B5	Without a gallery, distribution stays one-to-one	cmp	Counter	BR-S-022 – 025
B6	OAuth default is "unrestricted access to everything", labelled Recommended; 28 complaint topics	obs, voc	Avoid	BR-S-013 – 016
B7	Onboarding sells outcomes for five steps before asking one permission	obs	Steal	BR-S-010 – 012
B8	Deprecations broke running agents; the fix was customer labour	voc	Avoid	BR-S-026
C1	"Could really use a log" is the #1 organic request by engagement	voc	Build first	BR-M-013 – 017
C2	Blocked runs recorded as Completed; Failed and Approval tabs never fire	obs	Invert	BR-M-001 – 008
C3	Run output hides in per-run chat threads; the ledger carries only cost and status	obs	Invert	BR-M-015 , BR-M-018 – 022
C4	The best cost surface in the product is a settings page nobody finds	obs	Invert	BR-M-022 , BR-C-017

#	FINDING	TYPE	VERDICT	REQUIREMENTS
C5	Two cost surfaces disagree by exactly 3 on all three triggers	obs-n=1	Avoid	BR-M-035
C6	7 / 19 / 33 / 37–85 credits per run; one agent exhausts a month in 5–11 days	msr	Avoid	BR-C-017 , BR-M-027 – 032
C7	One pre-action cost estimate exists, on a knowledge-sync panel	obs	Steal	Law L3
C8	Trigger deletion behind an unlabelled ellipsis; six passes to find	obs	Avoid	BR-M-023 – 026 , BR-X-027 – 028
C9	Home surfaces accrued state and pulls the user back	obs	Steal	BR-M-042
C10	A 17-operation evals API; 1 of 6,377 posts mentions "evaluation"	voc	Gate it	BR-C-032 – 033 , BR-C-042
C11	Approvals conditioned on the arguments of the action; Slack as the surface	cmp	Steal	BR-X-008 – 011 , BR-X-005
C12	What a switcher gives up is an inspectable run	cmp	Answer	BR-M-018 – 022
D1	Per-role caps designed correctly, priced above the pain; 5 of 9 churn stories are billing	voc	Avoid	BR-M-032
D2	Background meter bills every 1–3 hours whether the product is used or not	msr	Avoid	BR-M-036
D3	Connector depth is the third-largest complaint family, 226 topics	voc	Avoid	\$10
D4	Three practitioners exited the canvas for the mass buyer within a year	cmp	Confirm	\$10
D5	Skills demand unproven: ~3 of 708 topics mention "skill"	cmp	Caveat	\$8.6

Reliability of this evidence. One operator, one free-tier account, four days compressed into roughly seven hours, n=1 throughout. The measurements are real and the arithmetic reproduces; the base rates do not exist. Nothing in this document should be defended on the grounds that a competitor does or does not do it — the findings are here because each one identifies a failure mode worth designing against, not because that company is the standard.

Appendix B · Screen inventory

The routes in `versions/v4/bench.html` . New and changed in v4 are marked.

Member routes

ROUTE	SCREEN	V4
<code>#/m/today</code>	Today — the feed	
<code>#/m/approvals</code>	Approvals	
<code>#/m/waiting</code>	Waiting on you	new
<code>#/m/thread/<id></code>	Agent thread	
<code>#/m/store</code>	Agent store	
<code>#/m/agent/<id></code>	Agent detail	
<code>#/m/installed</code>	My agents	
<code>#/m/impact</code>	What changed for me	
—	Install modal	changed

Team admin routes

ROUTE	SCREEN	V4
<code>#/a/pulse</code>	Team pulse	changed
<code>#/a/agents</code>	Agents	
<code>#/a/requests</code>	Requests	
<code>#/a/runs</code>	Run log	new
<code>#/a/run/<id></code>	Run trace	new
<code>#/a/schedules</code>	Schedules & budget	new
<code>#/a/new</code>	Create an agent	new
<code>#/a/build</code>	Agent builder — 5 tabs	changed
<code>#/a/test</code>	Test lab	
<code>#/a/publish</code>	Publish & rollout	changed
<code>#/a/share</code>	Share & ownership	new
<code>#/a/analytics/<id></code>	Agent analytics	
<code>#/a/context</code>	Business context	

Super admin routes

ROUTE	SCREEN	V4
<code>#/s/overview</code>	Organisation	
<code>#/s/agents</code>	All agents	
<code>#/s/people</code>	People	
<code>#/s/teams</code>	Teams & rollout	
<code>#/s/connections</code>	Connections & audit	
<code>#/s/spend</code>	Spend & governance	new

Developer handoff routes

ROUTE	SCREEN	V4
<code>#/h/spec</code>	Object model	changed
<code>#/h/flows</code>	Key flows	changed
<code>#/h/open</code>	Open questions	changed

Appendix C · Glossary

Agent — the only object a user authors. Instructions, scopes, triggers, guardrails, an input schema and a required outcome metric.

Ask — the object a run writes when it stops and needs a person. Question, why it did not decide, options, assignee, escalation time.

Auto-act — the autonomy level at which an agent performs an action without a human. Requires a written reason, an owner, a ceiling and a super admin's grant.

Backfill — a read-only run against the installer's own data, shown before consent and fired once on install.

Band — the cost unit. Light, standard or heavy. The only meter in the product.

Blocked — a run that stopped and asked. Never recorded as completed.

Business context — a team's own description of how it works, attached to its agents automatically.

Custody — which team owns an agent. Moves to the team on first publish and never returns.

Dry run — a draft executed against real data with every write suppressed.

Ignored — a run item with no response by the agent's next run. Distinct from dismissed: ignored is a design failure, dismissed is a judgement call.

Maintainer — the person responsible for a live agent. Gets its failures, budget warnings, requests and proposals.

Nothing to do — a run that completed and found nothing worth the person's attention. Three in a row is a setup problem.

Outcome metric — the one number an agent claims to move, read from a source system. Required to publish.

Proposal — a change an agent suggests to its own config. Enters the normal test and publish pipeline; never self-applies.

Run — one execution of one agent for one person. The unit of cost, of history and of debugging.

Trace — the step-by-step record of a run, including the steps where it decided not to act.

Waiting on you — the member surface where blocked runs appear, so that no question lives only inside a thread.